

Asignatura: Fundamentos de algoritmia

Evaluación continua. Cuestionario.

Grupo E.
Profesor: Isabel Pita.

Nombre del alumno:

1. Indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justifícala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
int n; std::cin >> n;
for (int i = 1; i < n-1; ++i)
    for (int j = i-1; j < i+1; ++j)
        x += v[i][j];
}
```

2. Indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justifícala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
int n, m; std::cin >> n >> m;
int i = n;
while (i > 0) {
    for (int j = 0; j < m; ++j) x += 2;;
    i = i / 2;
}
```

3. Escribe un predicado para expresar que todos los valores de un vector v son diferentes.
4. Escribe un predicado para indicar que los valores de un vector v están ordenados en orden creciente.
5. Dada la especificación:

$P:\{ a.size() \geq 0 \wedge b.size() \geq 0 \wedge ordenado(a) \wedge ordenado(b) \wedge diferentes(a) \wedge diferentes(b) \}$
resolver(vector<int> a, vector<int> b) dev vector<int> sol
 $Q:\{ (\forall k : 0 \leq k \leq sol.size() : sol[k] \in a \wedge sol[k] \in b) \wedge (\forall k : 0 \leq k \leq a.size() \wedge a[k] \in b : a[k] \in sol) \wedge$
 $(\forall k : 0 \leq k \leq b.size() \wedge b[k] \in a : b[k] \in sol) \}$

donde *ordenado* y *diferentes* son los predicados dados anteriormente y $v[k] \in a$ es un predicado que indica que el elemento $v[k]$ pertenece al vector a .

Y dada la siguiente implementación:

```
std::vector<int> resolver(std::vector<int> const& a, std::vector<int> const& b) {
    std::vector<int> sol;
    int i = 0; int j = 0;
    while (i < a.size() && j < b.size()) {
        if (a[i] < b[j]) ++i;
        else if (a[i] > b[j]) ++j;
        else { // son iguales
            sol.push_back(a[i]);
            ++i; ++j;
        }
    }
    return sol;
}
```

Indica un invariante del bucle que nos permita probar la corrección del mismo:

6. Da una función cota que permita probar la terminación del bucle de la función anterior.
7. Describe un algoritmo *óptimo* para eliminar todas las ocurrencias de un elemento de un vector ordenado con elementos repetidos, e indica su complejidad.

8. Indica cuál es la recurrencia que permite calcular el coste en el caso peor del algoritmo de búsqueda binaria. Haz el despliegue e indica el coste final del algoritmo.
9. Indica cuál es la recurrencia que permite calcular el coste en el caso peor del siguiente algoritmo y el coste final del algoritmo.

```
int f ( std::vector<int> & v, int ini, int fin){  
    if (ini == fin) return 0; // Vector vacio  
    else {  
        int m = (ini + fin - 1) / 2;  
        int x1 = f(v,ini,m+1);  
        int x2 = f(v,m+1,fin);  
        return x1 + x2;  
    }  
}
```

10. Indica cuál es la recurrencia que permite calcular el coste del algoritmo quicksort si el pivote se encuentra siempre cerca de la mediana del vector (el vector se divide siempre casi por la mitad). Indica el coste final del algoritmo.